ASP.NET Core Program.cs – Complete Explanation

This document explains every line inside `Program.cs` in ASP.NET Core using simple Tamil + English explanation.

Full Flow First (Big Picture)

```
Program.cs
   ↓
Services Register ஆகும்
   ↓
Application Build ஆகும்
   ↓
Middleware Pipeline உருவாகும்
   ↓
Routes Map ஆகும்
   ↓
Server Start ஆகும்
```

Full Program.cs

```
using SwiggyAPI.API.Services;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllers();

builder.Services.AddApplicationServices();

var app = builder.Build();

// Configure the HTTP request pipeline.
app.UseHttpsRedirection();

app.UseAuthorization();

app.MapControllers();
```

```
app.Run();
```

1️⃣ using SwiggyAPI.API.Services;

```
using SwiggyAPI.API.Services;
```

Why?

இந்த namespace உள்ள classes use பண்ணுறதுக்கு.

Here:

- `ServiceCollectionExtensions`

class use ஆகுது.

Without this?

Compiler error வரும்:

```
AddApplicationServices not found
```

Because compilerக்கு அந்த extension method எங்க இருக்குனு தெரியாது.

Real Meaning

Like importing tools box.

2️⃣ WebApplication.CreateBuilder(args)

```
var builder = WebApplication.CreateBuilder(args);
```

What this does?

இது ASP.NET Core application-ஐ configure பண்ணும் main builder object create பண்ணுது.

இதுல:

- DI Container
- Logging
- Configuration
- Middleware setup
- Kestrel server
- Environment variables

எல்லாமே initialize ஆகும்.

Internally

இதுக்குள்ள:

- IServiceCollection create ஆகும்
- IConfiguration load ஆகும்
- appsettings.json read ஆகும்
- Logging setup ஆகும்

Without this?

Application start ஆகாது.

Because இது தான் application foundation.

Real World Analogy

House build பண்ணுறதுக்கு foundation போடுற மாதிரி.

3️⃣ builder.Services.AddControllers();

```
builder.Services.AddControllers();
```

Why?

MVC Controllers register ஆகுது.

Meaning:

- API controllers use பண்ணலாம்
- `[ApiController]`
- `[HttpGet]`
- `[HttpPost]`

work ஆகும்

Internally என்ன நடக்கும்?

ASP.NET Core scans:

- Controllers folder
- Controller classes

Then DI containerல register பண்ணும்

Without this?

Controller APIs work ஆகாது.

Example:

```
[ApiController]
public class RestaurantController : ControllerBase
```

Request வந்தா:

```
404 Not Found
```

because controllers registered இல்லல.

Real Purpose

API endpoints activate பண்ணும்

4️⃣ builder.Services.AddApplicationServices();

```
builder.Services.AddApplicationServices();
```

Why?

Custom services DI containerல register ஆகுது.

Example:

```
services.AddScoped<IRestaurantService, RestaurantService>();
```

Meaning

Whenever app needs:

```
IRestaurantService
```

give:

```
RestaurantService
```

Without this?

Dependency Injection fail ஆகும்

Example:

```
public RestaurantController(IRestaurantService service)
```

Runtime error:

```
Unable to resolve service for type IRestaurantService
```

Real Purpose

Loose coupling + clean architecture.

5️⃣ var app = builder.Build();

```
var app = builder.Build();
```

Why?

இதுவரை configure பண்ணின எல்லாத்தையும் வைத்து final application object create ஆகுது.

Before Build()

You are only configuring.

After Build()

Application ready.

Internally

Creates:

- Middleware pipeline
- Service provider
- Application host

Without this?

App உருவாகாது.

No server.

No request processing.

Real Meaning

Construction complete → building ready.

6️⃣ app.UseHttpsRedirection();

```
app.UseHttpsRedirection();
```

Why?

HTTP requests automatically HTTPSக்கு redirect ஆகும்

Example

User hits:

```
http://localhost:5000
```

Automatically:

```
https://localhost:5001
```

Why Important?

HTTPS:

- encrypted communication
- secure data transfer
- prevents hacking/sniffing

Without this?

Site works in HTTP.

Security risk:

- passwords visible
- tokens exposed
- MITM attacks possible

Real Purpose

Security.

7️⃣ app.UseAuthorization();

```
app.UseAuthorization();
```

Why?

User க்கு access permission இருக்கா check பண்ணும்.

Example

```
[Authorize]
public IActionResult GetOrders()
```

Only logged-in users access.

Without this?

Authorization attributes ignored.

Protected APIs become public.

Important

This only checks authorization.

Authentication usually also needed:

```
app.UseAuthentication();
```

Real Purpose

Security + access control.

8️⃣ app.MapControllers();

```
app.MapControllers();
```

Why?

Controller routes activate ஆகும்

Example

```
[HttpGet("restaurants")]
```

maps URL:

```
/api/restaurants
```

Without this?

Routes register ஆகாது.

All APIs return:

```
404 Not Found
```

Real Meaning

Connect URL → Controller Method.

9️⃣ app.Run();

```
app.Run();
```

Why?

Application start ஆகும்

Kestrel server starts listening requests.

Internally

Starts:

- HTTP server
- request pipeline
- middleware execution

Without this?

Application exits immediately.

No API running.

Real Meaning

Engine start.

COMPLETE FLOW VISUAL

```
CreateBuilder()
    ↓
Register Services
    ↓
Build Application
    ↓
Configure Middleware
    ↓
Map Routes
    ↓
Run Server
```

Middleware Pipeline Concept

இந்த lines:

```
app.UseHttpsRedirection();
app.UseAuthorization();
```

middleware pipeline.

Request flow:

```
Request
   ↓
HTTPS Check
   ↓
Authorization Check
   ↓
Controller
   ↓
Response
```

Senior Developer Notes

Services Section

```
builder.Services
```

Used for:

- DI registrations
- app dependencies

Middleware Section

```
app.UseXXX()
```

Used for:

- request/response processing

Endpoint Section

```
app.MapControllers()
```

Used for:

- routing

MOST IMPORTANT UNDERSTANDING

Part	Purpose
builder.Services	Register dependencies
builder.Build()	Create app
app.UseXXX()	Middleware pipeline
app.MapControllers()	Route mapping

Part	Purpose
app.Run()	Start server

Real Enterprise Flow

Production apps additionally have:

```
app.UseAuthentication();
app.UseCors();
app.UseSwagger();
app.UseExceptionHandler();
app.UseStaticFiles();
```

Each middleware handles one responsibility.

This is called:

Middleware Architecture

ASP.NET Core's biggest strength.